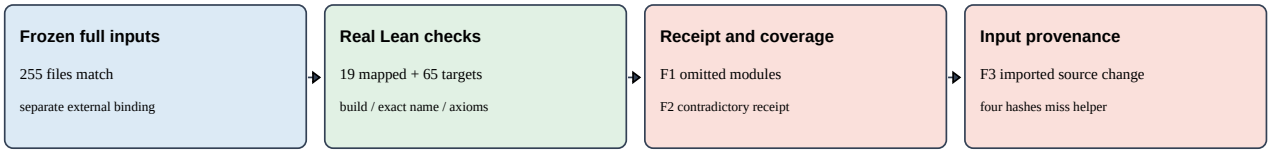
Independent Lean workflow and evidence-gate review



J3: passing kernel checks do not establish coverage, authentic execution, source fidelity or scientific promotion.

TASK-20261018-dae80c · J3-lean-workflow-and-evidence-gates · 2026-10-18

Own-joint verdict: breaks. Fresh machine checks reproduce the 19 mapped declarations and the identities of 65 discharged frozen targets. Three reproducible instrument boundaries prevent a blanket workflow validation: omitted project modules, inconsistent success receipts, and changes to imported proof inputs. These findings do not refute any producer theorem or supply its semantic-fidelity review. No scientific status is changed.

Authority, inputs and actual scope

The authority is the precommitted handoff and review plan at review checkout `1a9d9f4f8a9f214b44c1b424a885051ff3f2846b`. The producer revisions are crypto `eccff5b6cd1d0231b8d64da099cea954180d9d53`, cryptanalysis `25eb01fb6d5d309992edc11e4b940696cc6b112d`, autoresearcher `5e2bd6532fcf00730d8b231b55fe9427b7ae471d`, and CryptoLean `68117089795313bc2ffe3a3b5f72052d5fa7cb91`.

All 255 input files match the frozen snapshot and original producer files. All 11 workflow-input hashes match the review checkout. Twenty-nine package observations match the manifests' exact Git revisions with no tracked changes. These are provenance checks, not mathematical evidence. See `artifacts/input-hash-verification.json` and `dependency-pin-checks.json`. At sealing, only the working review checkout CLI had diverged from its initial hash: the Coordinator confirmed a subsequent repair had started after the confirmed findings. The committed and frozen original CLI remained unchanged. A final direct load of that frozen CLI reproduces the failure (exit 1 before import mutation, exit 0 after); `frozen-cli-confirmation.json` binds the loaded source hash. This review does not assess the moving repair version.

I read the adapted canonical skills, adapters, routing/catalog implementation, actual worker/CLI/target runner, root generation, shared gate/audit/selftests, consumer check scripts, producer correspondence reports/maps and receipt summarizer. Ancillary CI sources were retrieved by immutable Git revision. Source reads and their depth are enumerated in the attestation and `artifacts/source-reads.json`. Hashing a file is not represented as a semantic reading of its mathematical content. No sibling report or message was read, and no subagent was used. The Coordinator's messages supplied scope/path clarifications only.

Producer claims are expressly partial: 19 named declarations with conditional/model limitations, 65/66 frozen targets, independent semantic review pending, and no claim that all historical research is proved. Stronger historical propositions and full source fidelity belong to other joints; they were not certified here.

What freshly reproduced

Copied dependency caches and study projects under `/tmp/j3-lean-review` supplied isolated build products. Original producer source files were never edited. Consumer `lake build`, `#print axioms`, and the pinned shared transitive audits pass for all five crypto and four cryptanalysis declarations. The autoresearcher build and project audit pass for 498 theorem declarations, followed by a harness-generated exact-name/theorem-kind/transitive-axiom audit of all ten new declarations. All use only standard foundations; none of these new proofs uses a compiler-trust exception.

I additionally built and audited exact names from all frozen specs with existing proof files: 64 in the main Lean v4.35.0-rc1 workspace and one in the scoped Lean v4.22.0 workspace. Both source scans are empty. `ncp-reachability` remains without a proof file. The original summarizer independently reproduces 66 targets, 65 machine-verified and one undischarged. This is a combined fresh build/name/audit replay, not 65 new per-task CLI receipts or a semantic review. Consumer toolchain is Lean v4.35.0. See `replay-commands.json`, the fresh build/audit logs, and `all-frozen-audit-commands.json`.

The explicit router selects review-evidence plus formal for review/formal, formal for formalize/formal, and run plus formal for run/formal. Adapters point to the canonical skills. The skills preserve Coordinator promotion authority and correctly distinguish compilation, fidelity and review. The router accepts declared intent/topic values; these checks do not test natural-language classification or model compliance.

J3-F1: a tracked, unimported bad module escapes the shared gate

At frozen `crypto-lean/tools/proof-gate.sh:82`, the project path runs ordinary `lake build`; lines 86–90 discover only existing built `.olean` modules. Source discovery does not require every tracked Lean module inside that project to be built.

The clean scratch project has `Root.lean` proving True and a tracked `Omitted.lean` containing `theorem omittedFalse : False := by sorry`. The library root imports only Root. The project gate exits **0**, printing **PASS, 1 theorem**. Gating the same Omitted.lean explicitly exits **1**. This is false success for claimed project-wide source coverage, not a kernel proof of False. CI's project-discovery route uses this same gate. Logs: `shared-omitted-module.stdout.log`, `shared-omitted-receipt/`, and `shared-omitted-explicit.stdout.log`.

Smallest robust boundary: derive an explicit expected module/source set from tracked project sources, require every included module to build and be audited, and bind exclusions explicitly. Compare expected source coverage with fresh compiled module coverage. Merely auditing whichever oleans happen to exist cannot establish coverage; stale build outputs also need provenance.

J3-F2: summary accepts an internally contradictory success receipt

At frozen `crypto-autoresearcher/research/lean_retrospective_20261018/summarize.mjs:14–23`, success checks the top-level task theorem name, status/booleans and four hashes. It does not require the verification result's theorem identity to match, or an empty forbidden-construct list, even though `FormalProofResult.machine_verified` requires that list to be empty.

In an isolated copy, one affine-normal-form receipt was changed to `verification.theorem_name = Wrong.name`, `forbidden_constructs = [sorry:forged]`, empty build/audit logs and a bogus Lean version. Task identity and four hashes were retained. The unchanged summary exits **0**, still reports **65/66**, and labels that target machine-verified. A changed main proof source correctly exits **1** and reduces the accepted count to 64. See `forged-receipt.json`, `summary-forged-result.json`, and the three `summary-*.stdout.log` files. This demonstrates a receipt acceptance defect, not forgery in the original producer artifacts.

Smallest robust boundary: validate receipt schema and all task/result identities, required exact booleans, empty forbidden constructs, and provenance consistency against the frozen target. More fundamentally, arbitrary JSON is an assertion, not execution evidence: bind a trusted archived verifier run, exact verifier/input revisions and raw logs, or rerun the verifier. Hash fields and plausible log strings alone cannot authenticate a run. Keep summary-only acceptance explicitly separate from freshly verified proof status.

J3-F3: imported source changes are outside CLI receipt binding

At `orchestration/formal/cli.py:237–258`, before/after hashing covers only the spec, named theorem file, toolchain and dependency manifest. Imported local sources, lakefile, project audit and verifier source are absent. Pin syntax is checked, but actual dependency checkout identity/cleanliness is not checked there.

The timing control runs every subprocess against real Lean. A wrapper around the real version query performs a source mutation after the CLI takes its initial hashes. Changing the named theorem file yields exit **3** and an input-change diagnosis, as intended. Changing an imported helper yields exit **0** and machine-verified.

The stronger control starts with imported `def P : Prop := False` and unchanged target `theorem good : P := by trivial`: ordinary verification exits **1**. The timing wrapper changes only imported P to True; verification then exits **0**, with the same four provenance hashes as the failing run. Lean has checked the changed True proposition; it has not proved False. The failure is inability to bind verification to the original full input set. See `cli-loadbearing-before.json`, `cli-stale-loadbearing-import.json`, and `stale-import-{before,after,target}.lean`.

Smallest robust boundary: verify an immutable complete input snapshot, or hash and recheck the entire relevant source/configuration/audit set and exact clean dependency trees. Record its manifest digest in the receipt, along with verifier/gate revision and compiled-input provenance. A Git HEAD plus four hashes does not bind dirty or imported

input bytes. The external 255-file frozen manifest supplies a stronger boundary for this review; it does not retroactively enlarge historical CLI receipts.

Controls and failure classifications

Real worker controls reject wrong names and data-only declarations through the generated exact-theorem audit. Sorry and a plain user axiom fail the source scan. A private user axiom that evades that lexical pattern still fails the transitive target audit. Fake project output saying the audit passed cannot make a missing named theorem pass. The worker explicitly builds a target omitted from the project root: a valid omitted target passes and an invalid one fails.

An empty project audit file plus a real valid target passes the worker because the separate generated target audit actually checks that theorem. This is not empty proof acceptance, but the project-wide audit contents are not authenticated by the worker. The shared audit's data-only fixture fails for no theorems. The seven shared fixtures pass their expected signatures, including rejected custom axiom, build error, empty audit, unlisted native_decide and transitive sorry. One deliberately approved native fixture is reported TRUST, not kernel-only.

The actual target runner, configured for the scratch repository while retaining the real worker/CLI, passes one valid plus one missing target, fails a zero-verified run at its default floor, and fails a wrong theorem name. Missing source is intentionally undischarged; therefore a passing default floor of one never means every frozen target is proved. Existing receipt output is refused with exit **2**. See `additional-controls.log`, `final-controls.log`, per-control JSON and retained control sources.

Two initial controls were operational failures and are retained without counting them as axiom-audit evidence: an unregistered Hidden module failed to import, and an attribute control lacked its tactic extension. The later private-axiom control exercises the intended boundary. The first shared selftest failed because the byte-only frozen mirror lost executable bits; restoring the original Git executable mode in scratch makes all seven fixtures pass. The initial crypto CI blob retrieval hit an unavailable proxy; a subsequent local immutable-object read succeeds. No failed attempt was rewritten.

Limits, disposition and reproduction

Machine status for the named checked declarations is passed; mathematical fidelity is not reviewed by J3. Workflow fidelity is partial/mismatch at the three demonstrated boundaries, so workflow review outcome is needs_revision and own-joint verdict breaks. This is neither an overall approval vote nor a theorem-refutation verdict. No official ledger, proof source, gate, historical receipt or prior report was changed. No commit was made.

The installed autoresearch console and lean4checker are absent. Python entry points were exercised directly, and no external `lean4checker --fresh` claim is made. Full live CI, CI scope-script internals, every historical crypto project, proof generation/MathCode and hostile compiler/build-system compromises were not tested. Full review-independence composition is left to the Coordinator; reading sibling attestations to run it here would violate this handoff's blindness.

Reproducers are retained as `artifacts/control_suite.py`, `additional_controls.py`, `final_controls.py`, `replay.py` and `all_frozen_audit.py`. They declare their scratch paths and exact commands. With fresh scratch and output directories, run in that order using `PATH=/workspace/.elan/bin:$PATH ELAN_HOME=/workspace/.elan XDG_CACHE_HOME=/workspace/.cache PYTHONDONTWRITEBYTECODE=1 python3 <script>`. The fresh output paths in the scripts must be changed for a rerun; existing immutable CLI receipts intentionally cause exit 2. The replay script requires copies of the two verified producer cache trees named in it. Scratch Git init/add supplies tracked-file discovery; no scratch commit is required.

The diagram and current PDF describe these instrument boundaries. No mathematical route, performance graph or scientific canonical graph changes as a result of J3. Corrections should be a new producer revision, followed by fresh affected-joint checks, preserving this failed-control record.